



Kubernetes

พื้นฐาน (Basics)



แนะนำ Container Orchestration สำหรับนักพัฒนาและ DevOps

Containers

Pods

Services

Deployments

Scaling

สิ่งที่จะได้เรียนรู้

Agenda

01

Kubernetes คืออะไร?

ปัญหาที่แก้ได้ และสถาปัตยกรรมหลัก

02

Components หลัก

Node, Pod, Service, Deployment

03

kubectl & YAML

วิธีใช้งานและเขียน Manifest

04

Networking & Storage

ClusterIP, Ingress, PVC

05

Scaling & Rolling Update

Auto-scaling และ Zero-downtime Deploy

Kubernetes คืออะไร?



- ✗ Container จำนวนมาก จัดการยาก
- ✗ App ล่ม ต้องมีคนมาเปิดใหม่เอง
- ✗ Scale ขึ้นลงตามโหลดไม่ได้
- ✗ Deploy ใหม่ต้องหยุดให้บริการ

Kubernetes

- ✓ จัดการ Container อัตโนมัติ
- ✓ Self-healing: รีสตาร์ทเองได้
- ✓ Auto-scaling ตามการใช้งาน
- ✓ Rolling update ไม่ Downtime

สถาปัตยกรรม Kubernetes

Control Plane (Master)

API Server

รับคำสั่งทุกอย่าง

etcd

เก็บ state ของ cluster

Scheduler

เลือก Node สำหรับ Pod

Controller Manager

ดูแล desired state



Worker Node

kubelet

ดูแล Pod บน Node

kube-proxy

จัดการ networking

Container Runtime

รัน Container (Docker/containerd)

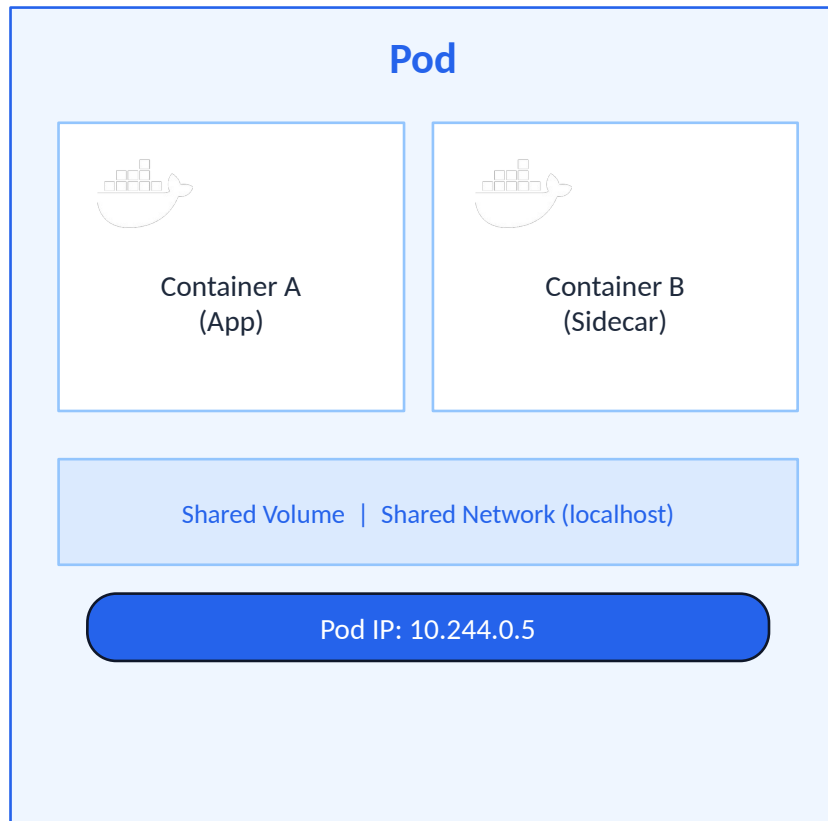
Pods

กลุ่ม Container ที่ทำงาน

02 Components หลัก: Pod

Pod คืออะไร?

- หน่วยพื้นฐานที่เล็กที่สุดของ Kubernetes
- บรรจุ Container ตั้งแต่ 1 ตัวขึ้นไป
- มี IP address เป็นของตัวเอง
- Container ใน Pod ใช้ Network และ Storage ร่วมกัน
- มีอายุชั่วคราว (ephemeral) — หากตายจะถูกสร้างใหม่



Deployment & Service

Deployment

- ควบคุมจำนวน Pod replicas
- Rolling update / rollback ได้
- ระบุ image version ของ container
- เป็น abstraction เหนือ ReplicaSet

Pod 1

Pod 2

Pod 3

Service

ClusterIP

เข้าถึงภายใน cluster เท่านั้น (default)

NodePort

เปิด port บน Node เพื่อเข้าจากภายนอก

LoadBalancer

สร้าง External Load Balancer (Cloud)

03 kubectl & YAML Manifest

คำสั่ง kubectl ที่ใช้บ่อย

```
kubectl get pods
```

ดูรายการ Pods

```
kubectl get nodes
```

ดูรายการ Nodes

```
kubectl describe pod <name>
```

ดูรายละเอียด Pod

```
kubectl apply -f file.yaml
```

สร้าง/อัปเดต resource

```
kubectl delete -f file.yaml
```

ลบ resource

```
kubectl logs <pod>
```

ดู logs ของ Pod

```
kubectl exec -it <pod> sh
```

เข้า shell ใน Pod

```
apiVersion: apps/v1
kind:
Deployment
metadata:

  name:
my-app
spec:

  replicas:
3
  selector:

    matchLabels:

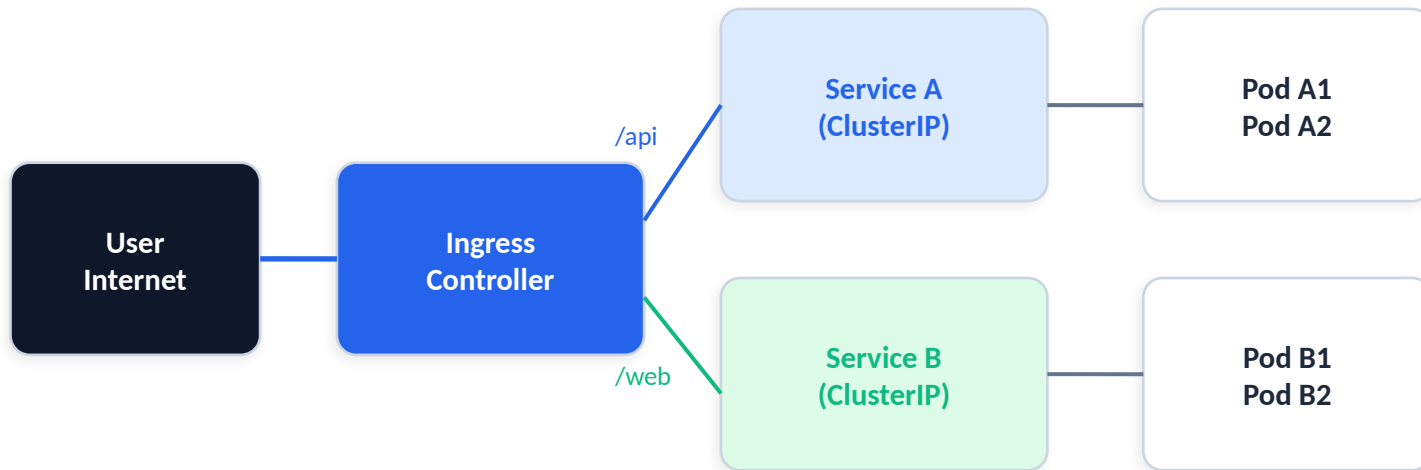
      app:
my-app
  template:

    spec:

      containers:

        - name:
web
          image: nginx:1.25
```

04 Networking: Ingress



Ingress จัดการ

- URL routing
- TLS/HTTPS
- Path-based
- Host-based
- Auth
- Rate limit

Storage: PV & PVC



PersistentVolume (PV)

พื้นที่เก็บข้อมูลจริงๆ ที่ Admin จัดเตรียมไว้
(NFS, Cloud Disk, Local Storage)



PersistentVolumeClaim (PVC)

คำขอใช้พื้นที่จาก Developer
กำหนดขนาด และ Access Mode



StorageClass

กำหนดประเภทของ Storage
รองรับ Dynamic Provisioning

PVC ← bind → PV

Pod mount PVC เพื่อใช้งาน Storage

05 Scaling & Rolling Update

Horizontal Pod Autoscaler (HPA)

เพิ่ม/ลด Pod ตามการใช้งาน CPU/Memory

Low Load



Med Load



High Load

kubectl autoscale deployment my-app \

--cpu-percent=60 --min=1 --max=10



Rolling Update



ก่อน Deploy (4
Pods v1)



เริ่ม Rolling (1/4)



กลาง Rolling (2/4)



เสร็จสมบูรณ์ (4/4)

ไม่มี Downtime ✓

ConfigMap & Secret



ConfigMap

เก็บ configuration ที่ไม่ sensitive เช่น
ค่า ENV, URL ของ services, config files

ใช้ inject เข้า Pod ผ่าน:

- Environment Variables
- Volume Mount
- Command-line args



Secret

เก็บข้อมูล sensitive เช่น
Password, API Key, TLS Certificate

จัดเก็บแบบ Base64 encoded
ควรใช้ร่วมกับ RBAC และ Encryption
at Rest เพื่อความปลอดภัย

สรุปสิ่งที่เรียนรู้



Architecture

Control Plane + Worker Node ทำงานร่วมกัน



Deployment

จัดการ Replicas, Rolling Update



Ingress

HTTP routing เข้า Cluster



Pod

หน่วยเล็กที่สุด บรรจุ Container



Service

Expose Pod: ClusterIP, NodePort, LB



Storage

PV, PVC, StorageClass

ขั้นถัดไป: Helm Charts | RBAC | Monitoring (Prometheus) | Service Mesh (Istio)